



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Mars Climate Orbiter

Ingeniería de Requisitos

Grado en Ingeniería Informática - UNIZAR 2025/2026

Autores:

Imad Habib Jali

Jorge Bellido Lobera

Daniel Blasco Labarta

901421@unizar.es

903080@unizar.es

900037@unizar.es

1. Descripción de la Experiencia

La misión Mars Climate Orbiter, lanzada el 11 de diciembre de 1998, fue diseñada para ser la primera estación meteorológica interplanetaria. Su objetivo era estudiar la atmósfera de Marte y servir como repetidor de comunicaciones para misiones posteriores.

El desarrollo fue una colaboración entre la NASA (Jet Propulsion Laboratory - JPL) y Lockheed Martin Astronautics. El sistema se dividía en dos componentes críticos cuyos requisitos debían estar perfectamente alineados: por un lado, el Segmento de Vuelo, el software a bordo de la sonda, encargado de ejecutar las órdenes de navegación; y por otro, el Segmento de Tierra, el software de procesamiento de datos (SM_FORCES) que calculaba el impulso necesario para corregir la trayectoria mediante las maniobras de "Desaturación de Ruedas de Reacción" (AMD).

2. Resultado de la Experiencia

El 23 de septiembre de 1999, tras 9 meses de viaje, la sonda inició la maniobra de inserción orbital. El motor principal se encendió, pero la comunicación se perdió permanentemente 4 minutos después.

El veredicto técnico establece que la sonda se acercó a Marte a una altitud de 57 km, cuando la altitud mínima de supervivencia era de 80 km (y la planificada de 140-150 km). La fricción atmosférica desintegró la nave. El costo total del desastre fue de 125.2 millones de dólares. El error fue una discrepancia de unidades, el software de tierra operaba en unidades imperiales (libra-fuerza segundo), mientras que el de vuelo esperaba unidades métricas (Newtons-segundo).

3. ¿Dónde se aplicó o se debería haber aplicado la IR?

La Ingeniería de Requisitos falló catastróficamente en la Especificación de Interfaces y en el Análisis de Dominio. La IR debería haber incidido en:

Requisitos de Interfaz de Software (SIR): Definir no solo el flujo de datos, sino el contrato de software. No se especificó el tipo de dato físico (unidades) en los Documentos de Control de Interfaz (ICDs).

Gestión de Trazabilidad Vertical: No hubo un hilo conductor que vinculara el requisito de navegación de alto nivel (Sistema Internacional) con la implementación de bajo nivel del contratista.

Falta de un Modelo de Datos Común: La IR falló al no establecer un "diccionario de datos" universal que eliminara la ambigüedad entre los distintos equipos de ingeniería.

4. Importancia de la aplicación y su impacto en el resultado

En este proyecto, la aplicación de la IR no era solo importante, era existencial.

Grado de importancia: Crítico. En sistemas complejos, la interoperabilidad es un requisito no funcional de primer orden.

Diferencia entre Verificación y Validación: El software de Lockheed fue verificado y funcionaba "bien" según sus requisitos locales (en unidades imperiales).

Validación: El sistema total falló la validación, ya que no cumplía con la necesidad global de la misión de navegar de forma segura.

Efecto: La ausencia de una validación de requisitos de unidades convirtió un sistema funcional en un proyectil mal dirigido. El software era técnicamente correcto de forma aislada, pero sistémicamente erróneo.

5. Relación entre el Producto Final y la IR

Existe una relación directa de causa-efecto donde un fallo en un requisito de bajo nivel invalidó millones de líneas de código y años de ingeniería. Aunque no hubo oportunidad de iterar sobre el producto físico, la NASA se vio obligada a iterar sobre su propio proceso de ingeniería de requisitos para misiones posteriores. Esta evolución incluyó el rediseño de protocolos para integrar tests de extremo a extremo, la ejecución de auditorías de interfaz en todas las misiones en curso y la implementación de estándares de contratación que exigen el uso del sistema métrico de forma obligatoria para eliminar cualquier margen de interpretación.

6. Opinión : ¿Qué y cómo se debería haber hecho?

Desde la perspectiva de la ingeniería de software moderna, el error fue un fallo de gobernanza y comunicación que la IR podría haber mitigado mediante acciones:

Diccionario de Datos Estricto: Se debería haber definido un glosario de términos y unidades vinculantes para todos los contratistas.

Pruebas de Interfaz (Shadow Testing): No basta con el éxito de las pruebas unitarias. Se debió simular la trayectoria real usando los datos de salida del software de tierra meses antes del lanzamiento en un entorno de integración.

Cultura de Escucha Activa y Gestión de Excepciones: Los navegantes notaron anomalías durante el viaje. Un proceso de IR maduro debe permitir que una observación técnica active una auditoría de requisitos inmediata, algo que se ignoró por exceso de confianza.

Tipado Fuerte y Análisis Dimensional: Utilizar herramientas que detecten inconsistencias en los tipos de datos (por ejemplo, evitar que el compilador permita sumar libras y newtons) habría identificado el bug en la fase de desarrollo, mucho antes de llegar a Marte.